

Optimizer Field Guide

Created by Lior Benshoshan

The optimizer takes an openMotor `.ric` file, searches the buildable variations of it against a set of limits, and returns motors that can be opened in openMotor and machined to the numbers given. This guide covers the interface panel by panel.

Start it

```
cd "Rocket Optimization"
python3 app.py # opens http://localhost:8420
```

That command is the entire install. On the first run it asks for confirmation before building an environment: a `.venv` in the project folder, the packages listed in `requirements.txt`, and a clone of openMotor pinned to a known commit. Nothing outside the project folder is modified. The requirements are **Python 3.9 to 3.13** and `git`. A C++ compiler is needed on macOS and Linux, because scikit-fmm publishes Windows wheels only and is built from source everywhere else:

`xcode-select --install` on macOS,
`apt install build-essential python3-dev` on Debian. Separately, openMotor contains one compiled module that a BATES simulation never calls; if that one fails to build, setup substitutes a pure-Python stand-in.

WHY 3.13 AND NOT NEWER

The pinned wheels stop there. `requirements.txt` carries two sets: Python 3.10 and later get numpy 2 and the versions built for 3.13, while 3.9 keeps numpy 1, the last release that supports it. PyPI records no upper bound on any of them, so on an unsupported version pip attempts a source build rather than refusing, and that build needs a C and Fortran toolchain. Setup checks the range before installing anything, and will use a supported interpreter if one is installed alongside a newer one.

3.14 is not reachable yet. scipy and scikit-image both require 3.11 or later at the versions that support it, which is workable, but scikit-fmm publishes no 3.14 wheel at any version and its source does not build against numpy 2 before 2025.6.23.

On start the application loads whatever `.ric` file is in `motor/`. No filename is significant. The folder ships empty, so there is nothing to optimise until a motor is placed there.

`python tests/sample_motor.py motor/sample.ric` writes a generic one for experimenting. **Change** in the header selects a different file. A loaded motor is simulated exactly as its file specifies; the application never modifies it.

THE SHORT VERSION

Load a motor, select which dimensions may change and give each a step, choose what to optimise, set the limits, press **Optimize**, select a design from the curve, and export the `.ric`. The rest of this guide details those steps.

The steps, in order

1

LOADED MOTOR

Check the loaded motor

Grains, bore, nozzle and propellant, along with the motor's performance as it stands: thrust, impulse, peak pressure, Kn and mass flux. Warnings from openMotor appear here as well. An over-expanded nozzle is reported as *low exit pressure*.

These numbers are worth reading first. A motor that already breaks a limit as loaded is not a small problem to correct, and the search will be looking for a substantially different motor.

2

FIXED HARDWARE

Confirm the case

Grain outer diameter, length and count are read from the file and shown greyed out. These represent the case and the mould, so they are neither optimised nor editable here. They can be changed in openMotor and the file reloaded.

Inhibited ends is the one live control in this panel, being a casting choice rather than a hardware constraint. Where a value has been overridden, the panel reports **modified from file** and offers **Reset to file**.

3

WHAT MAY CHANGE

Unlock dimensions and set the machining grid

One row per dimension: six grain cores, nozzle throat, nozzle exit and throat length. Unticking a row holds that dimension at its current value and removes it from the search entirely, rather than searching it over a zero-width range.

Step sets the machining grid. It accepts decimals and shop fractions, so `0.05`, `1/16`, `1 1/16` and `2-1/2` are all valid. Left blank, the optimiser returns arbitrary values such as a 2.4713" core. The grid is anchored at zero, so a 1/16" step yields whole sixteenths rather than sixteenths measured from the lower bound.

The buttons below the table set every step at once. The default is `0.05"`. `0.01"` is two decimal places, which is the finest tolerance commonly held.

4

POSSIBLE CONFIGURATIONS

See how big the problem is

This panel updates as the configuration changes. It counts *distinct motors* rather than grid points, since cores are stored sorted and one set of six diameters therefore represents one motor rather than 720. The figure reaches the order of 10^{13} for nine free dimensions on a 0.05" grid, and 10^{19} on a 0.01" one, which is why the application uses a genetic search rather than enumeration.

Where a Kn limit is set, the panel also reports what the limits provably exclude and offers **Apply tighter bounds**, which narrows the search to the region where a legal motor can still exist. Nothing legal is removed.

5

WHAT TO OPTIMIZE

Choose the objectives

Any of sixteen metrics, each set to maximise, minimise or reach a target value. Selecting two produces a trade-off curve rather than a single answer. Thrust and impulse oppose each other, so the useful question is usually which point on that curve to build.

Presets are provided for the common cases: *Max initial thrust*, *Max impulse*, *Flattest curve* and *Thrust vs impulse*.

6

DESIGN CONSTRAINTS

Set the envelope

Rows of *metric* $\leq$ or $\geq$ *value*. The defaults are peak chamber pressure 500 psi, peak Kn 225, peak mass flux 1.05 lb/in²s and port/throat ratio 1.4, which are amateur-practice numbers rather than the case rating a `.ric` file usually carries. Unticking a row ignores that limit, and **+ add** introduces further ones. Two limits on the same metric are flagged, since only the tighter one binds.

Below the table, the loaded motor is shown against every limit, both as a table and as its pressure, Kn and mass flux over time with the limits drawn on as dotted lines.

No design is reported unless it satisfies every ticked limit when re-simulated at full fidelity with all safety margins removed.

Choose an ordering rule

Four options. The choice has a material effect on the result:

Any order	No rule. Tends to choke the aft grain.
Never narrow aft	Cores do not shrink going aft, ties allowed. This is the default.
Strictly wider	Each core exceeds the one ahead by a set minimum. Costs roughly 0.8% of thrust compared with allowing ties.
Paired mandrels	A small number of sizes shared across grains, such as three sizes across six grains. Requires fewer mandrels.

Set the budget and the number of searches

Four presets set the simulation budget and the number of independent searches together.

PRESET	BUDGET	SEARCHES	PER SEARCH	POPULATION × GENERATIONS
Quick	2,400	2	1,200	40 × 30
Standard	4,800	3	1,600	40 × 40
Thorough	10,000	5	2,000	40 × 50
Extreme	24,000	8	3,000	60 × 50

These figures come from a measurement on a nine-dimension, two-objective configuration at a 0.05" step. The best motor stopped improving a little past 2,000 simulations, and independent searches stopped paying at about five. Increasing the number of searches is generally more effective than increasing the budget, up to that point.

The boxes behind the presets are read-only until *I know what I'm doing* is ticked, which unlocks the budget, the search count and the core count.

Surrogate search samples the space, trains models on those simulations, and searches the models rather than openMotor. It is cheaper and faster and it maps the whole trade-off curve, but its answers come from a model and are not guaranteed to be the best available, which is why it is marked beta. The full search runs every candidate through openMotor.

Cores to use sets how many simulations run at once. Automatic uses the machine's performance cores where the platform reports them, since the slowest worker sets the pace and adding efficiency cores makes a run longer rather than shorter.

No time estimate is shown until **Run the diagnostic** has timed real simulations at these settings. A rate the program has not measured on this machine is a guess. The measured figure is then increased by 20% to account for the machine slowing down as it heats up.

Reading the results

Four profiles are available across the top of the workspace, presenting the same run from different angles.

ONE MOTOR

Design Review

Thrust curve, chamber pressure and Kn, a scaled cross-section, the spec sheet, and margin bars showing proximity to each limit. Amber indicates a design within 3% of a limit.

ALL OF THEM

Trade-off Explorer

The curve of options, where any point can be clicked to load it into Design Review. Also shows every design attempted, a parallel-coordinates view of what the successful designs have in common, and a table with a `.ric` export button on each row.

THE SEARCH

Optimizer Diagnostics

Progress against simulations spent and which limit is binding. In trade-off mode it also reports model accuracy and which dimension has the greatest influence.

BEFORE AND AFTER

Compare & Safety

The loaded motor against the selected design, a specification delta table, mass flux in each grain against its limit, and a tornado chart of what most affects the leading objective.

IDENTIFYING THE BINDING LIMIT

When a result is disappointing, the bar chart in Diagnostics is the panel to consult. It reports what share of legal designs sit within 2% of each limit. A bar near zero indicates a limit that is not constraining the search. A red bar indicates the limit worth revisiting.

Exporting and reporting

Every row of the options table carries a `.ric` export button. The exported file opens in openMotor and re-simulates to exactly the figures displayed, as nothing shown in the interface is a model prediction.

Each completed run writes its own report to `reports/report.pdf` automatically. The report covers hardware, limits, the trade-off curve, tabulated options and geometry, all derived from the run itself. A run that finds no legal design is documented in the greatest detail, covering which limit could not be met, how close the search came, and what would have to change.

Design sheets gives one PDF per design on the curve, with the dimensions to machine to, packaged as a zip archive.

Motor files gives one RASP `.eng` per design in a zip, the format OpenRocket and RockSim read. Each curve is a real openMotor run at the verification timestep, so an `.eng` agrees with the figures this run reported for the same design. One caveat matters: RASP has a single total-mass field and this application never models hardware, so that field holds the propellant mass alone, and the diameter and length are the grain outer diameter and the grain stack. The case, nozzle and closures have to be added before the file is used for an altitude simulation. Every file repeats this in its own header.

Definitions

Initial thrust	The mean of the thrust curve over the first 0.35 s, excluding openMotor's zero-thrust sample at $t = 0$. This is neither peak thrust nor thrust at ignition.
Legal	Re-simulated at a 0.002 s timestep with every search-time margin removed, and satisfying each ticked limit on those figures.
Kn and pressure	Effectively one constraint, since chamber pressure is a monotone function of Kn for a given propellant. Setting both means one of the two binds and the other has no effect. The application reports the Kn at which the pressure limit is reached, so the redundant limit can be identified.
Seeds	The search is stochastic, and identical configurations have produced results several percent apart on best thrust. Several searches therefore run and merge by default. A single search should be treated as one sample of the answer.
Grain order	In openMotor's model, grain order affects only mass flux and port/throat ratio, and both are most favourable with the widest core aft. Cores are therefore stored sorted, removing a 720-fold redundancy from the search.

Running it without the app

The application is a thin layer over the library, so its behaviour can be scripted. `rocketopt.runner.run` takes a configuration and a motor and returns verified designs. `rocketopt.report` and `rocketopt.bundle` render those into PDFs.

```
scripts/verify_ordering.py      check the core-sorting assumption against the simulator
.venv/bin/python -m pytest tests/ -q
```

IF A RUN RETURNS NO DESIGNS

An empty result is itself a finding rather than a failure. Diagnostics reports which limit every candidate violated, and the report's *no answer* section covers the same ground in more detail. Where the nozzle is fixed, it gives the throat diameter that would permit a legal motor and demonstrates that no core diameter could have achieved one.

openMotor 0.6.2, GPLv3, cloned during setup rather than redistributed

Application at `http://localhost:8420`. Reports in `reports/`, run data in `outputs/`

Lior's Really Good™ Rocket Optimizer • created by Lior Benshoshan